Course Project

Apply NLP to Your Domain

MAI554 – Deep Learning for Natural Language Processing

Spring 2026 • Prof. Anis Koubaa

Alfaisal University – College of Engineering

1 What Is This Project?

In this project, your team will build a **working NLP application** that solves a real problem from your professional field. You will use the techniques you learned in class (tokenization, embeddings, RNNs, Transformers, etc.) to process text and produce useful results.

The Big Idea

Pick a problem you care about from your own field (medicine, finance, business, CS, etc.), find a relevant text dataset, and build an NLP system that solves it. Your diverse professional backgrounds are your greatest asset in this project.

1.1 Key Facts

Item	Detail
Weight	15% of your final grade
Team Size	2–3 students (solo allowed with reduced scope)
Deliverables	Proposal + Notebook + Report + Presentation
Proposal Due	**Wednesday April 22** (submit on AssessX)
Notebook + Report Due	**Wednesday May 13** (submit on AssessX)
Presentation	**Thursday May 21** (special session, 10 minutes per team)

2 Timeline

You have **6 weeks** from now to complete the project. Here is the schedule with exact dates:

Key Dates

- **Eid Al-Adha Break**: May 22 – June 1 (no classes)
- **All project work must be completed before the break.**
- Classes are every **Wednesday 5–8pm**.

Wk	Date	Milestone	What to Do
8	**Apr 10** (Fri)	Project assigned	Read this document. Form your team. Start brainstorming.
9	**Apr 15** (Wed)	Team finalized	Finalize team on AssessX. Identify your problem and dataset.
	Apr 22 (Wed)	**Proposal due**	Submit 1–2 page proposal on AssessX before class.
10	**Apr 29** (Wed)	Development	Collect data, preprocess, start coding your pipeline.
11	**May 6** (Wed)	Development	Train models, run experiments, iterate on results.
12	**May 13** (Wed)	**Notebook + Report due**	Submit final ZIP on AssessX (notebook + report + data).
13	**May 21** (Thu)	**Presentations**	Special session (replaces a missed lecture). 10-min team presentations + live demos.
	May 22 (Fri)	*Eid break begins*	*No more classes until June 2.*

3 What Do You Need to Build?

Your project must use **at least 2 techniques** from the course. We have organized the techniques into 4 levels. **Pick any 2 (or more)** from the table below:

Level	Technique	From Lectures	Example
1	Text Processing	Tokenization, Embeddings (L2–L3)	Build a tokenizer, compare BPE vs WordPiece, visualize embeddings
2	Sequence Model	RNN, LSTM, GRU (L4)	Train a sentiment classifier or text generator
3	Transformer	Fine-tune a pre-trained model (L5–L6)	Fine-tune BERT or GPT for your task
4	Full Application	Chatbot, Translation, RAG (L8–L10)	Build a Q&A system over documents

3.1 Minimum Technical Requirements

⚠ What "Using a Technique" Actually Means

Simply calling a library function (e.g., `pipeline("sentiment-analysis")`) is **not** "using a technique." You must demonstrate that you **understand** what the model does internally. The table below shows the minimum expected effort for each level.

Level	Not Enough (will score low)	What We Expect (minimum)
1	Call `tokenizer.encode()` and move on	Tokenize your dataset, **analyze** the vocabulary (size, most frequent tokens, OOV rate). **Compare** at least 2 tokenizers (e.g., BPE vs WordPiece) and explain which works better for your data and **why**. Visualize embeddings using t-SNE or PCA.
2	Copy-paste an LSTM tutorial from the internet	**Build and train** your own RNN/LSTM/GRU in PyTorch. Show the **training loss curve**. **Compare** at least 2 architectures (e.g., RNN vs LSTM) and explain the difference in performance using course concepts (vanishing gradients, gates, etc.).
3	Run `pipeline("task")(text)` in 3 lines	**Fine-tune** a pre-trained model (BERT, GPT, etc.) on **your own dataset**. Show training vs. validation loss. Report **before fine-tuning vs. after fine-tuning** performance. Explain **what the model learned** from your domain data.
4	Use ChatGPT through the web interface	Build a working pipeline with **your own code**: e.g., a RAG system with document chunking + embedding + retrieval + generation, or a chatbot with a custom prompt template. Show the system processing **at least 5 different inputs** with analysis of when it succeeds and fails.

3.2 Mandatory Deliverables in Your Notebook

Regardless of which levels you choose, your notebook **must** include all of the following:

Required in Every Project

1. **Baseline vs. Improved Model**: Train a simple baseline first (e.g., bag-of-words + logistic regression, or a pre-trained model without fine-tuning), then show how your chosen technique improves upon it. Report the **numerical difference** in performance.
2. **Error Analysis**: Show at least **5 examples** where your model fails and explain **why** it fails, using concepts from the course (e.g., "the LSTM fails on this long sentence because of the vanishing gradient problem").
3. **Training Curves**: Plot loss (and accuracy/F1 if applicable) over training epochs for all models you train. Discuss whether the model is underfitting or overfitting.

4. **Quantitative Results Table**: A single table comparing all your models/approaches with at least 2 metrics (e.g., accuracy, F1, precision, recall, BLEU).
5. **Architectural Explanation**: Include a paragraph (or diagram) explaining **how your model works internally**, using the vocabulary from the course (hidden states, attention weights, gates, embeddings, etc.). Do not just say "we used BERT" – explain *what* BERT does to your input text step by step.

For Students with Less Coding Experience

You can use libraries like **Hugging Face**, **spaCy**, or **LangChain**. But you must still fulfill the 5 mandatory requirements above. The goal is to show *understanding*, not just running code. If you use a library, add markdown cells explaining what each step does and connect it to what you learned in class.

For Students with Strong Coding Skills

Challenge yourself! Implement a model from scratch using PyTorch (e.g., build your own LSTM or a mini-Transformer). This demonstrates deeper understanding and will earn you higher marks in Technical Depth.

4 Project Ideas by Domain

You are free to choose any topic. Here are some ideas to get you started:

Your Field	Project Idea	Levels
Medicine	Extract diagnoses and medications from clinical notes (NER)	1 + 3
Finance	Sentiment analysis on financial news or earnings calls	1 + 2
Business	Customer review summarizer or FAQ chatbot	1 + 4
CS / AI	Build a mini-Transformer from scratch on a small dataset	2 + 3
Law	Classify legal documents by type or extract key clauses	1 + 3
Education	Automated essay grading or feedback system	2 + 3
Any field	Arabic–English translation using a fine-tuned model	1 + 4
Any field	Domain-specific Q&A system over PDF documents (RAG)	3 + 4
Any field	Fake news / misinformation detector	1 + 3
Any field	Resume parser and job-matching system	1 + 3

5 Deliverable 1: Proposal (Due April 22 – 3%)

Proposal – What to Submit

Submit a **1–2 page PDF** on Moodle containing:

1. **Team Members**: Names, student IDs, and professional backgrounds.
2. **Problem Statement**: What problem are you solving? Why does it matter in your field? (3–4 sentences)
3. **Dataset**: What text data will you use? Where will you get it? How large is it? *Examples: Kaggle datasets, Hugging Face datasets, company data (anonymized), web-scraped data, public APIs.*
4. **Approach**: Which 2+ levels will you use? Which specific models/techniques? Why?
5. **Expected Outcome**: What will your final system do? What does success look like?
6. **Week-by-Week Plan**: A simple 6-line table showing what each team member will do each week.

6 Deliverable 2: Jupyter Notebook (Due May 13 – 4%)

Notebook – What to Submit

Submit a **single Jupyter Notebook (.ipynb)** on Moodle. The notebook must be organized into these sections (use markdown headings):

1. **# 1. Introduction**
   Problem description, motivation, and which levels you implemented. (Markdown)
2. **# 2. Data Loading & Exploration**
   Load your dataset, show its size, display sample rows, and basic statistics. (Code + Output)
3. **# 3. Preprocessing**
   Tokenization, cleaning, train/test split. Show before/after examples. (Code + Output)
4. **# 4. Model Implementation**
   Build and train your model(s). This is the core of your project. Show the architecture, training loop, and loss curves. (Code + Output)
5. **# 5. Evaluation**
   Test your model on held-out data. Report metrics (accuracy, F1, BLEU, etc.). Show examples of correct and incorrect predictions. (Code + Output)
6. **# 6. Discussion**
   What worked? What didn't? Why? What would you do differently with more time? (Markdown)
7. **# 7. Conclusion**
   Summary of what you built and key findings. (Markdown)

⚠ Common Mistakes to Avoid

- Do **not** submit a notebook that doesn't run. Test it from top to bottom before submitting.
- Do **not** copy-paste code from the internet without understanding it. You will be asked about your code during the presentation.
- Do **not** forget to include your **dataset** (or a link to download it) so we can reproduce your results.
- Do **not** train on test data. Always keep a held-out test set.

7 Deliverable 3: Short Report (Due May 13 – 3%)

This deliverable trains you to write in the style of a scientific paper. You will summarize your project in a **2–3 page report** structured like a real research paper.

Report – What to Submit

Submit a **2–3 page PDF** (single-spaced, 11pt font, 2.5cm margins). Use the following structure:

1. **Title and Authors**
   Project title, team member names and affiliations (e.g., "MSc AI, Alfaisal University").
2. **Abstract** (100–150 words)
   A single paragraph summarizing the problem, approach, key results, and main conclusion. Write this *last*, after everything else is done.
3. **1. Introduction** (∼0.5 page)
   What problem are you solving? Why does it matter? What is your approach in one sentence? End with: "The rest of this paper is organized as follows. . . "
4. **2. Related Work** (∼0.25 page)
   Briefly describe 2–3 existing approaches to your problem. Cite them properly. Explain how your work differs or builds on them.
5. **3. Methodology** (∼0.5 page)
   Describe your pipeline step by step: dataset, preprocessing, model architecture, training setup. Include a simple **system diagram** (a flowchart showing Input → Processing → Output).
6. **4. Experiments and Results** (∼0.5 page)
   What metrics did you use? What were the results? Include **one table** with your main results (e.g., accuracy, F1, BLEU). Include **one figure** (e.g., training loss curve, confusion matrix, or example output).
7. **5. Discussion** (∼0.25 page)
   What worked well? What didn't? Why? What are the limitations?
8. **6. Conclusion** (∼0.25 page)
   Summarize your findings in 3–4 sentences. Suggest 1–2 concrete improvements for future work.
9. **References**
   List all sources you used (minimum 3 references). Use a consistent citation format (e.g., APA or IEEE).

Tips for Writing a Good Report

- Write in **third person** ("The model achieves...", not "I trained...").
- Every figure and table must have a **caption** and be **referenced in the text**.
- The Abstract should be self-contained – a reader who reads *only* the abstract should understand what you did and what you found.
- Keep it concise. A 2-page paper that says everything clearly is better than a 3-page paper with filler.

8 Deliverable 4: Presentation (Thursday May 21 – 2%)

Presentation – What to Do

Deliver a **10-minute presentation** followed by **5 minutes of Q&A**.

Special Session

Presentations will take place on **Thursday May 21** (not the regular Wednesday slot). This special session replaces a missed lecture. Time and room will be announced on Moodle. Attendance is **mandatory** for all students.

Suggested slide structure (max 10 slides):

1. **Title Slide**: Project name, team members
2. **The Problem**: What are you solving and why?
3. **The Data**: What dataset, how large, sample examples
4. **The Approach**: Which techniques (levels), why these choices
5. **Architecture Diagram**: How the system works (a simple flowchart)
6. **Live Demo**: Show the system working on a real input
7. **Results**: Key metrics, example predictions
8. **Lessons Learned**: What worked, what surprised you, what failed
9. **Conclusion**: Summary and future improvements

Rules:

- All team members **must** present.
- You **must** show a live demo (even a simple one).
- Be prepared to answer questions about your code and design choices.

9 Grading Rubric (15%)

Your project will be graded on the following criteria:

Criterion	Points	What We Look For
Problem Relevance	2	Clear, well-defined problem connected to a real domain. Appropriate dataset identified.
Technical Depth	3	Correct use of at least 2 course techniques (levels). You understand *why* each technique was chosen, not just *how* to call it.
Implementation	4	Clean, well-documented notebook. Code runs end-to-end. Results are reproducible.
Report Quality	3	Well-structured 2–3 page paper following the scientific format. Clear writing, proper figures/tables, references.
Analysis	1	Thoughtful discussion of results, limitations, and potential improvements.
Presentation	2	Clear 10-min delivery, effective live demo, confident Q&A responses. All members present.
Total	**15**	

10 Forming Your Team

- Teams of **2–3 students**. Solo projects are allowed but with a reduced scope (1 level instead of 2).
- **Mixed backgrounds are encouraged**. A team with a CS student and a medical student can build a stronger project than two CS students working alone – one provides the coding, the other provides the domain knowledge.
- Submit your team composition as part of the proposal (Week 9).

How to Find a Team

Post your background and project interests on the Moodle discussion forum. Look for complementary skills: if you're a strong coder, find a teammate with deep domain knowledge. If you're a domain expert, find a teammate who can help with the implementation.

11 Frequently Asked Questions

1. **Do I need to code everything from scratch?**
   No. You can use libraries like Hugging Face, spaCy, LangChain, or PyTorch. What matters is that you *understand* what the code does and can explain it.

2. **What if I have no coding experience?**
   Pair up with a teammate who codes. Your job is to define the problem, find the dataset, interpret the results, and present. Both contributions are equally valued.

3. **Can I use ChatGPT or Copilot to help write code?**
   Yes, but you must be able to **explain every line of code** during the presentation Q&A. If you cannot explain it, it will count against you.

4. **Where can I find datasets?**

   - Hugging Face Datasets – thousands of NLP datasets
   - Kaggle – community datasets with notebooks
   - Your own workplace data (must be anonymized)

5. **How will the demo work?**
   Run your Jupyter notebook live and show the model processing a new input. For example: type a sentence and show the model's prediction.

6. **What if my model doesn't work well?**
   That's OK! A project that honestly analyzes *why* the model failed and proposes improvements can score just as well as a project with perfect accuracy. We grade understanding, not just results.

7. **Can two teams choose the same topic?**
   Yes, as long as they use different datasets or approaches.

12 How to Submit

All submissions are through **AssessX** as a **group project**.

Submission via AssessX

1. You will receive an **email from AssessX** with instructions on how to create your group and join the project assignment.
2. **One member per team** creates the group on AssessX and adds the other members.
3. All deliverables are submitted as a **single ZIP file** named:
   `MAI554_Project_TeamName.zip`
4. Only **one submission per group** is needed (any team member can submit).

12.1 What Goes in the ZIP File

File	Description	Required
`proposal.pdf`	1–2 page proposal	Week 9
`notebook.ipynb`	Jupyter Notebook with all code and outputs	Week 13
`report.pdf`	2–3 page short paper	Week 13
`slides.pdf`	Presentation slides (max 10 slides)	Week 14
`data/`	Dataset files or a `README.txt` with download link	Week 13

12.2 Submission Checklist

Item	Due
Received AssessX email and created group	Apr 15
Proposal PDF submitted on AssessX	**Apr 22**
Dataset collected and preprocessed	Apr 29
Model trained and evaluated	May 6
Jupyter Notebook finalized (.ipynb)	**May 13**
Short report written (2–3 pages, PDF)	**May 13**
Final ZIP uploaded to AssessX	**May 13**
Presentation slides ready (max 10 slides)	**May 21**
Live demo tested and working	**May 21**

Good luck! Build something you're proud of.

Questions? Email: akoubaa@alfaisal.edu

Appendix: Example Project Walkthrough

This appendix shows a **complete example** of what a good project looks like, step by step. Use it as a reference for the level of depth and analysis we expect.

Project: Financial News Sentiment Analyzer

Team: 2 students (1 Finance background, 1 CS background)
Levels used: Level 1 (Tokenization + Embeddings) + Level 2 (LSTM) + Level 3 (Fine-tuned BERT)
Problem: Classify financial news headlines as *Positive*, *Negative*, or *Neutral* to help traders make faster decisions.

Step 1: Dataset (Sections 2–3 of Notebook)

- **Source:** "Financial PhraseBank" from Hugging Face (~4,800 labeled headlines).
- **Split:** 70% train (3,360), 15% validation (720), 15% test (720).
- **Exploration:** Show class distribution (Positive: 28%, Neutral: 59%, Negative: 13% — imbalanced!), average headline length (12 words), and 3 example headlines per class.

Step 2: Tokenization Analysis (Level 1)

This is where you show understanding, not just call a function.

What a Good Level 1 Looks Like

1. **Compare 2 tokenizers** on the same 5 headlines:
   - BPE (GPT-2 tokenizer): "stockholders" → ["stock", "holders"] (2 tokens)
   - WordPiece (BERT tokenizer): "stockholders" → ["stock", "##holder", "##s"] (3 tokens)
2. **Analyze vocabulary**: BPE produces 15% fewer tokens on financial text because it learned "stock" as a common merge. Report average tokens per headline for each tokenizer.
3. **Visualize embeddings**: Use t-SNE to plot Word2Vec embeddings of the top 50 financial terms. Show that "profit", "gain", "revenue" cluster together (positive), while "loss", "decline", "debt" form a separate cluster (negative).
4. **Conclusion**: "WordPiece produces more tokens for financial jargon, but BERT's pre-training compensates. We chose BERT's tokenizer because we will fine-tune BERT in Level 3."

Step 3: Baseline Model

Every Project Needs a Baseline

Before building anything fancy, train a **simple baseline** so you can measure improvement.

Baseline: TF-IDF + Logistic Regression (no deep learning).

- Vectorize headlines using TF-IDF (5,000 features).

- Train a logistic regression classifier.
- **Result:** Accuracy = 71%, F1 = 0.65
- This takes 10 lines of code and 5 seconds to train. It sets the bar.

Step 4: LSTM Model (Level 2)

What a Good Level 2 Looks Like

1. **Architecture**: Embedding (dim=100) → LSTM (hidden=128, 2 layers) → Dropout(0.3) → Linear(128 → 3) → Softmax. Draw a diagram.
2. **Training**: Adam optimizer, lr=0.001, 20 epochs, batch size=32, cross-entropy loss.
3. **Training curve**: Plot train loss and validation loss over 20 epochs. Note: "Validation loss stops decreasing at epoch 12 — the model starts overfitting after that. We use early stopping."
4. **Compare RNN vs LSTM**:
   - Vanilla RNN: Accuracy = 68%, F1 = 0.61 (worse than baseline!)
   - LSTM: Accuracy = 76%, F1 = 0.72 (better than baseline)
   - **Explanation**: "The RNN performs worse because financial headlines often have key information at the end (e.g., 'despite strong revenue growth, *net loss widened*'). The RNN's vanishing gradient causes it to forget the beginning. The LSTM's forget and input gates preserve this long-range dependency."
5. **Result**: LSTM improves over baseline by +5% accuracy and +0.07 F1.

Step 5: Fine-Tuned BERT (Level 3)

What a Good Level 3 Looks Like

1. **Pre-trained model**: `bert-base-uncased` from Hugging Face.
2. **Before fine-tuning**: Run BERT zero-shot on the test set. Accuracy = 58%. "BERT knows general English but not financial language."
3. **Fine-tuning setup**: Learning rate = 2e-5, 3 epochs, warmup steps = 100. Freeze all layers except the last 2 + classification head.
4. **After fine-tuning**: Accuracy = 87%, F1 = 0.84.
5. **Explanation**: "Fine-tuning taught BERT that 'revenue growth' is positive in financial context, whereas in general English it's neutral. The attention mechanism allows BERT to weigh 'loss' more heavily when it appears after 'net', distinguishing 'net loss' (negative) from 'loss prevention' (positive)."

Step 6: Error Analysis

This Is Where You Show Real Understanding

Show 5 examples where the best model (BERT) fails, and explain *why* using course concepts.

Headline	True	Predicted	Why It Failed
"Revenue rose but margins fell sharply"	Neg	Pos	Mixed sentiment; model fixates on "rose"
"The company announced a restructuring"	Neg	Neutral	Requires world knowledge that restructuring = layoffs
"Shares were unchanged"	Neutral	Neg	"unchanged" is rare in training data

Discussion: "Most errors occur on mixed-sentiment headlines or implicit sentiment. This is a known limitation of sequence classification — the model must compress an entire headline into a single label. A token-level approach (e.g., aspect-based sentiment) might handle mixed headlines better."

Step 7: Final Results Table

This is the table that goes in your report and presentation.

Model	Accuracy	F1	Precision	Recall	Training Time
TF-IDF + LogReg (baseline)	71%	0.65	0.67	0.64	5 sec
Vanilla RNN	68%	0.61	0.63	0.60	8 min
LSTM (2-layer)	76%	0.72	0.74	0.71	12 min
BERT (zero-shot)	58%	0.49	0.52	0.48	—
BERT (fine-tuned)	**87%**	**0.84**	**0.85**	**0.83**	25 min

Why This Project Would Score Well

What Makes This a Strong Project

1. **Problem Relevance (2/2)**: Clear domain problem (finance), real dataset, practical value.
2. **Technical Depth (3/3)**: Uses 3 levels (not just 2), compares tokenizers, compares RNN vs LSTM with course-concept explanations, fine-tunes BERT with before/after analysis.
3. **Implementation (4/4)**: Clean notebook, all code runs, training curves shown, reproducible.
4. **Report (3/3)**: Follows paper structure, includes results table and figure, proper references.
5. **Analysis (1/1)**: Error analysis with explanations, honest about limitations.
6. **Presentation (2/2)**: Live demo typing a headline and showing the prediction.

Estimated score: 15/15

What a Weak Version of This Project Looks Like

- Uses `pipeline("sentiment-analysis")` without fine-tuning and reports one number.
- No baseline comparison — impossible to know if the model is any good.
- No error analysis — just says "the model achieves 87% accuracy."
- No training curves — unclear if the model overfit.
- No architectural explanation — just says "we used BERT."
- **Estimated score: 4–6/15**